



Devoir Maison - Correction Evaluation de programme

Nicolas Vacelet - Université Marie et Louis Pasteur
Mars 2025

1 Machine de Turing

1.1 La machine

Alphabet : $\Gamma = \{R, P, H, C, Po, Bo, N, B\}$

Ensemble d'états : $Q = \{q_0, q_b, q_n, q_v, nqbl, qbv, qbn, \dots, q_{acc}, q_{rej}\}$

État	R	P	H	C	Po	Bo	N	B
q_0	q_b, x, R	q_b, x, R	q_l, x, R	q_l, x, R	q_v, x, R	q_v, x, R	q_n, x, R	
q_b			q_{bl}, x, R	q_{bl}, x, R	q_{bv}, x, R	q_{bv}, x, R	q_{bn}, x, R	
q_l	q_{bl}, x, R	q_{bl}, x, R			q_{lv}, x, R	q_{lv}, x, R	q_{ln}, x, R	
q_v	q_{bv}, x, R	q_{bv}, x, R	q_{lv}, x, R	q_{lv}, x, R			q_{vn}, x, R	
q_{bl}					q_{blv}, x, R	q_{blv}, x, R	q_{bln}, x, R	q_{acc}
q_{bv}			q_{blv}, x, R	q_{blv}, x, R			q_{bvn}, x, R	
q_{bn}			q_{bln}, x, R	q_{bln}, x, R	q_{bvn}, x, R	q_{bvn}, x, R		
q_{lv}	q_{blv}, x, R	q_{blv}, x, R					q_{lvn}, x, R	
q_{vn}	q_{bvn}, x, R	q_{bvn}, x, R	q_{lvn}, x, R	q_{lvn}, x, R				
q_{ln}	q_{bln}, x, R	q_{bln}, x, R			q_{lvn}, x, R	q_{lvn}, x, R		
q_{blv}							q_{blvn}	q_{acc}
q_{bln}					q_{blvn}	q_{blvn}		q_{acc}
q_{bvn}			q_{blvn}	q_{blvn}				q_{acc}
q_{lvn}	q_{blvn}	q_{blvn}						
q_{blvn}								q_{acc}

La machine utilise le symbole **x** pour marquer les éléments déjà traités sur le ruban. Toutes les combinaisons d'états possibles sont représentées de manière systématique :

- **q_b** : état avec une **base** unique (riz ou pâtes)
- **q_l** : état avec un **légume** unique (haricots ou carottes)
- **q_v** : état avec une **viande** optionnelle (poulet ou bœuf)
- **q_bl** : état avec une base et un légume
- **q_bv** : état avec une base et une viande
- **q_blv** : état avec base, légume et viande
- etc. (toutes combinaisons valides)

Cette machine couvre exhaustivement toutes les permutations possibles d'ingrédients tout en garantissant l'absence de doublons (chaque type d'ingrédient n'apparaît qu'au plus une fois). Les états d'acceptation sont ceux qui, après lecture complète de la liste, satisfont la présence d'au moins une base et d'un légume.

Notre machine est déterministe car à chaque combinaison (état courant, symbole lu) correspond au plus une seule transition définie.

Le problème est décidable car :

- La machine s'arrête toujours
- Le temps d'exécution est borné par $O(n)$

Un repas doit contenir soit du riz (**Ri**), soit des pâtes (**Pa**), mais pas les deux. La formule logique est :

$$(\mathbf{Ri} \vee \mathbf{Pa}) \wedge \neg(\mathbf{Ri} \wedge \mathbf{Pa})$$

Ce qui peut aussi s'écrire comme un OU exclusif (XOR) :

$$\mathbf{Ri} \oplus \mathbf{Pa}$$

Supposons qu'un repas valide doit contenir :

- Exactement une base (riz ou pâtes),
- Au moins un légume (**Carotte**, **Haricot**).

La formule est :

$$\mathbf{Valide} = (\mathbf{Ri} \oplus \mathbf{Pa}) \wedge (\mathbf{Carotte} \oplus \mathbf{Haricot})$$

Un repas végétarien ne contient aucune viande . La formule est :

$$\mathbf{Vegan} = \mathbf{Valide} \wedge \mathit{not}(\mathbf{Po} \vee \mathbf{Bo})$$

1.2 Complexité

L'objectif sous-jacent de cet exercice est d'illustrer que le problème de validation d'un repas, soumis à un ensemble de contraintes, peut être réduit à un problème bien connu en informatique théorique : le problème de satisfiabilité propositionnelle, appelé SAT.

En effet, chaque contrainte imposée sur les repas (par exemple : exactement une base, au plus une viande, sans gluten, végétarien, etc.) peut être exprimée sous forme de formules logiques propositionnelles. Les ingrédients deviennent des variables booléennes (présent ou non dans le repas), et les contraintes sont traduites en formules reliant ces variables.

Dès lors, la question "ce repas est-il valide ?" revient à déterminer si une certaine formule logique (la conjonction de toutes les contraintes) est satisfiable, c'est-à-dire s'il existe une combinaison d'ingrédients qui respecte toutes les règles. C'est précisément le problème SAT.

Cette réduction a une conséquence importante : toutes les propriétés connues du problème SAT s'appliquent également au problème de validation de repas. En particulier, dans le cas général (lorsque les contraintes sont arbitraires), le problème est **NP-complet**. Cela signifie qu'il est au moins aussi difficile que tout autre problème dans NP, et qu'il est peu probable qu'un algorithme polynomial existe pour le résoudre dans tous les cas.

1.3 Algorithme glouton

Conséquence pratique : recours à un algorithme glouton Bien que le problème général de validation d'un repas valide soit **NP-complet**, cela ne signifie pas qu'il est impossible à résoudre en pratique. En effet, la complexité NP-complète signifie que *dans le pire des cas*, aucun algorithme connu ne résout tous les cas en temps polynomial. Mais dans de nombreux cas concrets, des heuristiques ou des approches approximatives permettent d'obtenir des résultats satisfaisants.

C'est dans ce contexte qu'un **algorithme glouton** peut constituer une solution efficace. Un algorithme glouton prend des décisions locales optimales à chaque étape (par exemple, choisir un ingrédient admissible dès qu'il satisfait les contraintes en cours), sans revenir en arrière. Bien que cette approche ne garantisse pas une solution optimale dans tous les cas, elle est souvent très rapide et fournit des solutions valides dans des situations pratiques.

Dans le cas des repas, un algorithme glouton peut, par exemple :

- sélectionner une base admissible,
- ajouter un légume compatible,

- ajouter des ingrédients optionnels si cela ne viole pas les contraintes restantes.

Cette stratégie est simple à implémenter, rapide à exécuter, et donne des résultats corrects tant que les contraintes ne sont pas trop conflictuelles ou interdépendantes. Ainsi, malgré la complexité théorique du problème, une approche gloutonne permet de le résoudre efficacement dans un cadre applicatif.

- Pour n ingrédients, chaque étape nécessite de vérifier $O(n)$ ingrédients.
- Chaque vérification de contrainte prend $O(1)$ si les contraintes sont simples.
- La complexité totale est $O(n^2)$ dans le pire cas.
- Optimalité non garantie
- Dépendance à l'ordre : Le résultat peut varier selon l'ordre de traitement.
- solution non garantie

2 Complexité

Pour analyser la complexité de la fonction `convert` en nombre de divisions entières (/), examinons son comportement :

1. Chaque appel à `convert` effectue exactement **1 division entière** ($n/10$).
2. La fonction appelle récursivement `convert` avec $q = n/10$ jusqu'à ce que $q == 0$.
3. Le nombre d'appels (et donc de divisions) est égal au nombre de chiffres de n en base 10.
4. Le nombre de chiffres d'un entier n en base 10 est donné par :

$$\lfloor \log_{10} n \rfloor + 1$$

La complexité en nombre de divisions entières est $O(\log n)$.

Exemple : Pour $n = 1234$ (4 chiffres), la fonction effectue 4 divisions :

- $1234/10 = 123$
- $123/10 = 12$
- $12/10 = 1$
- $1/10 = 0$ (condition d'arrêt).

Ainsi, la complexité est bien logarithmique en fonction de la taille de n .

3 Calculabilité

Dans cet exercice, on ne demande pas de démonstration, juste de donner un argument.

Question 1. Soient L_1 et L_2 deux problèmes indécidables. Est-ce que $L_1 \cap L_2$ peut être décidable? Pourquoi?

Prenons l'exemple de L_1 et L_2 indécidables et complètement disjoints. Dès lors, $L_1 \cap L_2 = \emptyset$ et par définition $\emptyset$ est décidable (trivial). Ainsi, $L_1 \cap L_2$ peut être décidable.

Question 2. Est-ce que un problème NP-complet peut être indécidable? Pourquoi?

un problème est dans la classe NP si il existe une machine de Turing non-deterministe en temps polynomial qui décide ses instances. Donc tout problème NP est par définition décidable. Les problèmes NP-complets sont "en quelque sorte" les plus compliqués de la classe NP, donc tout problème NP-Complete est forcément décidable. (voir cours)

Question 3. Soient L_1 et L_2 deux problèmes décidables. Est-ce que $L_1 \cap L_2$ peut être indécidable? Pourquoi?

Par définition, si L_1 et L_2 sont deux problèmes décidables, alors leur union et intersection sont décidables. Donc $L_1 \cap L_2$ ne peut pas être indécidable.

Comme L_1 est décidable, il existe une machine de Turing M_1 , qui décide l'ensemble des mots de L_1 . Comme L_2 est décidable, il existe une machine de Turing M_2 , qui décide l'ensemble des mots de L_2 .

Pour cela, on peut imaginer une machine de Turing M , qui combine M_1 et M_2 sur deux rubans différents. M s'arrêtant forcément dès que M_1 et M_2 s'arrêtent. Une telle machine M , reconnaît alors à la fois les mots communs de L_1 et de L_2 .